10장 자연어 처리를 위한 임베딩

10.1 임베딩

임베딩이란 사람이 사용하는 언어(자연어)를 컴퓨터가 이해할 수 있는 숫자 형태인 벡터로 변환한 결과를 의미한다. 임베딩은 다음과 같은 역할을 한다.

1. 단어 및 문장 간의 관련성을 계산한다.
2. 의미적 또는 문법적 정보를 함축한다.

임베딩 방법은 다음과 같이 네 가지로 구분할 수 있다.

- 희소 표현 기반 임베딩
- 횟수 기반 임베딩
- 예측 기반 임베딩
- 횟수/예측 기반 임베딩

10.1.1 희소 표현 기반 임베딩

희소 표현이란 대부분의 값이 0으로 채워져 있는 표현 방법을 의미한다. 대표적인 희소 표현 방법이 원-핫 인코딩이다.

원-핫 인코딩은 주어진 텍스트에서 단어 집합의 크기를 벡터의 차원으로 설정하고, 표현하고자 하는 단어의 인덱스에만 1을 부여하며 나머지 인덱스에는 모두 0을 부여하는 방식으로 단어를 벡터화한다.

원-핫 인코딩은 다음과 같은 두 가지 한계를 갖는다.

1. 단어 벡터끼리 내적했을 때 결과가 0이 되어 단어 간 의미적 관련성을 전혀 표현하지 못한다. 즉, 모든 단어가 서로 독립적인 관계로 표현된다.
2. 단어 집합의 크기가 커질수록 벡터의 차원이 함께 커지는 차원의 저주 문제가 발생한다.

이러한 한계를 극복하기 위해 워드투벡터, 글로브, 패스트텍스트와 같은 인공 신경망 기반의 임베딩 방법이 등장했다.

10.1.2 횟수 기반 임베딩

횟수 기반 임베딩은 단어가 문서에 등장하는 빈도를 이용하여 단어를 벡터로 표현하는 방식이다. 대표적으로 카운터 벡터와 TF-IDF가 있다.

카운터 벡터

카운터 벡터는 문서 집합에서 단어를 토큰으로 생성하고, 각 단어의 출현 빈도수를 이용하여 인코딩한 벡터이다. 불용어를 지정하면 의미가 크지 않은 단어를 제외하고 벡터를 생성할 수 있다.

TF-IDF

TF-IDF는 단어의 빈도와 역 문서 빈도를 사용하여 단어가 문서 집합 내에서 갖는 중요도를 계산하는 통계적 수치이다.

TF(단어 빈도)는 특정 단어가 하나의 문서 내에서 등장하는 빈도를 의미한다. 값이 높을수록 문서 내에서 중요한 단어일 가능성이 높아진다.

DF(문서 빈도)는 특정 단어가 등장하는 문서의 개수를 의미한다.

IDF(역 문서 빈도)는 DF에 반비례하는 값으로, 전체 문서 수를 해당 단어가 등장한 문서 수로 나눈 값에 로그를 취하여 구한다. 여러 문서에 공통적으로 자주 등장하는 단어일수록 IDF 값은 작아지며, 특정 문서에서만 등장하는 단어일수록 IDF 값은 커진다. 특정 단어가 모든 문서에 한 번도 등장하지 않을 경우 분모가 0이 되는 문제를 방지하기 위해 분모에 1을 더하는 스무딩을 적용하기도 한다.

TF-IDF는 TF와 IDF를 곱하여 계산하며, 다음과 같은 경우에 사용할 수 있다.

1. 문서에서 특정 단어가 얼마나 중요한지 나타내고자 할 때
2. 불용어처럼 중요도가 낮으면서 자주 등장하는 단어를 걸러내고자 할 때
3. 키워드 추출 등 텍스트 마이닝에서 단어에 가중치를 부여하고자 할 때

10.1.3 예측 기반 임베딩

예측 기반 임베딩은 신경망 구조 혹은 모델을 이용하여 특정 문맥에서 어떤 단어가 나올지 예측하면서 단어를 벡터로 만드는 방식이다. 대표적으로 워드투벡터가 있다.

워드투벡터

워드투벡터는 신경망 알고리즘으로, 주어진 텍스트에서 텍스트의 각 단어마다 하나씩 일련의 벡터를 출력한다. 출력 벡터가 2차원 그래프에 표시될 때, 의미론적으로 유사한 단어의 벡터는 서로 가깝게 표현된다. 이때 서로 가깝다는 의미는 코사인 유사도를 이용하여 단어 간의 거리를 측정한 결과로 나타나는 관계성을 의미한다. 즉, 워드투벡터를 이용하면 특정 단어의 동의어를 찾을 수 있다.

워드투벡터는 일정한 크기의 윈도우로 분할된 텍스트를 신경망의 입력으로 사용하며, 분할된 텍스트는 대상 단어와 컨텍스트의 쌍으로 네트워크에 공급된다. 워드투벡터는 학습 방식에 따라 CBOW와 skip-gram으로 나뉜다.

CBOW

CBOW는 단어를 여러 개 나열한 후 이와 관련된 단어를 추정하는 방식이다. 즉, 문장에서 등장하는 n개의 단어열에서 다음에 등장할 단어를 예측한다. CBOW 신경망에서 입력층과 은닉층 사이의 가중치, 은닉층과 출력층 사이의 가중치는 모든 단어에서 공통으로 사용된다.

skip-gram

skip-gram은 CBOW 방식과 반대로 특정한 단어에서 문맥이 될 수 있는 단어를 예측하는 방식이다. 즉, 중심 단어에서 주변 단어를 예측한다. 이때 입력 단어 주변에서 문맥으로 고려하는 단어의 개수를 윈도우 크기라고 한다.

CBOW와 skip-gram 중 어느 알고리즘이 더 우수하다고 일률적으로 결론 내리기보다는, 분석하고자 하는 데이터의 성격과 분석의 목적을 종합적으로 고려하여 필요한 방식을 선택해야 한다.

10.1.4 횟수/예측 기반 임베딩

앞서 살펴본 횟수 기반과 예측 기반 임베딩의 단점을 보완하기 위한 임베딩 기법에는 대표적으로 패스트텍스트와 글로브가 있다.

패스트텍스트

패스트텍스트는 워드투벡터의 단점을 보완하고자 개발된 임베딩 알고리즘이다. 워드투벡터는 분산 표현을 이용하여 단어를 벡터로 표현하기 때문에, 사전에 없는 단어에 대해서는 벡터 값을 얻을 수 없고 자주 사용되지 않는 단어에 대해서는 학습이 불안정하다는 한계가 있다.

패스트텍스트는 문서의 각 단어를 n-그램으로 표현하여 이러한 한계를 보완한다. n 값에 따라 단어가 여러 개의 부분 단어로 분리되며, 학습이 완료된 후에는 데이터셋의 모든 단어를 각 n-그램에 대해 임베딩한다. 따라서 사전에 없는 단어가 등장하더라도 n-그램으로 분리된 부분 단어와의 유사도를 계산하여 의미를 유추할 수 있으며, 등장 빈도수가 적은 단어라도 n-그램 단위로 참고할 수 있는 경우의 수가 많아지므로 상대적으로 학습의 안정성이 높아진다.

글로브

글로브는 횟수 기반의 LSA와 예측 기반의 워드투벡터 단점을 보완하기 위한 모델이다. 글로브는 단어에 대한 글로벌 동시 발생 확률 정보를 포함하는 단어 임베딩 방법으로, 단어에 대한 통계 정보와 skip-gram 방식을 결합한 방식이라고 할 수 있다. 이를 통해 단어 간의 관련성을 통계적인 방법으로 표현할 수 있다.

10.2 트랜스포머 어텐션

어텐션은 주로 언어 번역에서 사용되기 때문에 인코더와 디코더 네트워크를 사용한다. 입력에 대한 벡터 변환은 인코더에서 처리하고 모든 벡터를 디코더로 보낸다. 모든 벡터를 전달하는 이유는 시간이 흐를수록 초기 정보를 잃어버리는 기울기 소멸 문제를 해결하기 위해서이다.

하지만 모든 벡터가 전달되면 행렬의 크기가 매우 커지는 단점이 있으며, 이를 해결하기 위해 소프트맥스 함수를 사용하여 가중합을 구하고 그 값을 디코더에 전달한다. 디코더는 은닉 상태에 대해 중점적으로 집중해서 보아야 할 벡터를 소프트맥스 함수로 점수를 매긴 후 각각을 은닉 상태의 벡터들과 곱하고, 이 은닉 상태를 모두 더해 하나의 값으로 만든다. 즉, 어텐션은 모든 벡터 중에서 꼭 살펴보아야 할 벡터들에 집중하겠다는 의미이다.

트랜스포머는 어텐션을 극대화하는 방법으로, 어텐션에서는 인코더와 디코더에 네트워크가 하나씩 있었던 반면 트랜스포머는 인코더와 디코더를 여러 개 중첩시킨 구조를 사용한다. 이때 각각의 인코더와 디코더를 블록이라고 한다.

하나의 인코더 블록은 셀프 어텐션과 전방향 신경망으로 구성된다. 인코더에서는 단어를 벡터로 임베딩하며, 이를 셀프 어텐션과 전방향 신경망으로 전달한다. 셀프 어텐션은 문장에서 각 단어끼리 얼마나 관계가 있는지를 계산해서 반영하며, 여기서 파악된 단어 간 관계는 전방향 신경망으로 전달된다.

디코더 블록은 셀프 어텐션, 인코더-디코더 어텐션, 전방향 신경망의 세 개 층으로 구성된다. 인코더에서 넘어온 벡터가 처음 만나는 것이 셀프 어텐션 층이며, 이후 인코더-디코더 어텐션 층에서는 인코더가 처리한 정보를 받아 어텐션 메커니즘을 수행하고, 마지막으로 전방향 신경망으로 데이터가 전달된다.

어텐션 메커니즘은 다음과 같은 과정으로 수행된다.

1. 디코더의 현재 시점에서 단어를 예측하기 위해, 인코더의 모든 은닉 상태 값이 디코더의 현 시점 은닉 상태와 얼마나 관련이 있는지를 나타내는 어텐션 스코어를 계산한다.
2. 어텐션 스코어에 소프트맥스 함수를 적용하여 시간의 가중치로 변환한다.
3. 시간의 가중치와 인코더 은닉 상태 값의 가중합을 계산하여 컨텍스트 벡터를 구한다.
4. 컨텍스트 벡터와 디코더 이전 시점의 은닉 상태 및 출력을 이용하여 디코더의 다음 은닉 상태를 구한다.

어텐션이 적용되면 컨텍스트 벡터가 시점마다 다르게 계산되는 반면, 어텐션이 적용되지 않은 경우에는 컨텍스트 벡터가 고정되어 있다는 차이가 있다.

10.2.1 seq2seq

seq2seq는 입력 시퀀스에 대한 출력 시퀀스를 만들기 위한 모델이다. seq2seq는 품사 판별과 같은 시퀀스 레이블링과는 차이가 있다. 시퀀스 레이블링은 입력 단어가 여러 개 주어지면 그에 대응하는 동일한 개수의 출력을 만드는 형태로, 입력과 출력의 문자열 길이가 같다. 반면 seq2seq는 번역에 초점을 둔 모델로, 입력 시퀀스와 의미가 동일한 출력 시퀀스를 만드는 것이 목표이며 입력과 출력 단어 간의 일대일 관계는 중요하지 않다. 또한 입력과 출력 시퀀스의 길이도 서로 다를 수 있다.

seq2seq는 인코더와 디코더로 구성된다. 인코더는 입력 문장을 단어별로 순서대로 인코딩하며, 문장의 끝에는 이를 표시하는 토큰이 붙는다. 인코더는 임베딩 계층과 GRU 계층으로 구성되며, 임베딩 계층에서 입력을 벡터로 변환한 뒤 GRU 계층에서 연속하여 들어오는 입력을 계산하고 이전 계층의 은닉 상태를 바탕으로 게이트를 갱신한다.

디코더는 인코더의 출력을 디코딩하여 다음 출력을 예측하며, 임베딩 계층, GRU 계층, 선형 계층으로 구성된다. 임베딩 계층에서는 출력을 위한 딕셔너리를 조회할 테이블을 만들고, GRU 계층에서는 다음 단어를 예측하기 위한 확률을 계산하며, 선형 계층에서는 계산된 확률 값 중 최적의 값을 선택하기 위해 소프트맥스 활성화 함수를 사용한다.

모델 학습 시에는 티처포스라는 기법이 많이 사용된다. 티처포스는 번역하려는 목표 단어를 디코더의 다음 입력으로 넣어 주는 기법으로, 학습 초기에 안정적인 훈련이 가능하며 기울기를 계산할 때 빠른 수렴이 가능한 장점이 있지만 네트워크가 불안정해질 수 있는 단점도 있다.

일반적인 seq2seq 모델은 인코더에서 사용하는 순환 신경망의 마지막 은닉 상태만 디코더로 전달되기 때문에 입력 문장이 긴 시퀀스일 경우 정확한 처리가 어렵다. 이는 다음과 같은 두 가지 이유에서 비롯된다.

1. 하나의 고정된 크기의 벡터에 모든 정보를 담다 보니 정보의 손실이 발생한다.

2. 순환 신경망에서 발생할 수 있는 기울기 소멸 문제가 발생한다.

어텐션 메커니즘은 디코딩 프로세스 동안 입력 시퀀스의 모든 은닉 상태를 유지하고 활용하기 때문에 이러한 정보 손실과 기울기 소멸 문제를 완화할 수 있다. 다만 어텐션 메커니즘의 성능 향상 효과는 시퀀스가 길수록 더 크게 나타난다.

10.2.2 버트(BERT)

버트는 기존의 단방향 자연어 처리 모델들의 단점을 보완한 양방향 자연어 처리 모델이다. 검색 문장의 단어를 입력된 순서대로 하나씩 처리하는 것이 아니라, 트랜스포머를 이용하여 구현되었으며 방대한 양의 텍스트 데이터로 사전 훈련된 언어 모델이다.

버트의 기본 구조는 트랜스포머의 인코더를 쌓아 올린 구조로, 주로 문장 예측을 할 때 사용한다. 튜닝(미세 조정)을 통해 사전 학습된 버트 위에 분류를 위한 신경망을 한층 추가하는 방식을 사용한다. 즉, 버트는 트랜스포머와 사전 학습을 사용하여 성능을 향상시킨 모델이다.

버트의 학습 절차는 다음과 같다.

1. 문장을 버트의 입력 형식에 맞게 변환한다. 이때 문장의 시작은 [CLS], 문장의 끝은 [SEP]로 표시한다.
2. 한 문장의 단어들에 대해 토큰화를 진행한다. 이때 단어는 가장 작은 조각 단위로 쪼개질 수 있다.
3. 각 토큰에 대해 고유의 아이디를 부여한다. 토큰이 존재하지 않는 자리는 0으로 채운다.

버트 모델은 전이 학습을 기반으로 하며, 이때 전이는 인코더-디코더로 된 모델 중 인코더만 사용하는 것을 의미한다. 기존의 인코더-디코더 모델들과 다르게 CNN, RNN을 이용하지 않고 어텐션 개념을 도입했다는 특징이 있다.

버트는 BERT-base와 BERT-large라는 두 가지 버전으로 제공된다. 이때 L은 전이 블록의 개수, H는 은닉층의 크기, A는 전이 블록에서 사용되는 어텐션 블록의 개수를 의미한다. L, H, A가 클수록 블록을 많이 쌓았고 표현하는 은닉층의 크기가 크며 어텐션 개수를 많이 사용했다는 의미이다. BERT-base와 BERT-large는 학습 파라미터의 개수에서도 차이가 난다.

10.3 한국어 임베딩

한국어에 대한 임베딩도 영어에 대한 임베딩과 동일한 방법으로 수행할 수 있다. 즉, 임베딩 방법만 알고 있다면 언어와 상관없이 단어 및 문장에 대한 임베딩을 진행하고, 모델을 생성하고 훈련시킨 후 예측 및 분류를 수행할 수 있다.

다만 다국어를 지원하는 사전 훈련된 버트 모델을 한국어에 사용하더라도, 해당 토크나이저가 단어의 가장 작은 조각을 기준으로 토큰화하도록 설계되어 있기 때문에 한글 단어가 의미 단위와 다르게 과도하게 잘게 쪼개질 수 있다는 한계가 있다. 영어는 단어 기준으로 토큰화가 비교적 잘 이루어지지만, 영어가 아닌 언어의 경우 토큰화가 정확하지 않을 수 있다.

따라서 한국어에 대한 임베딩에서는 다국어 버트 모델보다 KoBERT와 같이 국내에서 한국어에 특화되어 개발된 모델을 이용하는 것이 정확도 측면에서 더 바람직하다.